서 울 대 학 교
SEOUL NATIONAL UNIVERSITY

Systems Software & Architecture Laboratory
Dept. of Computer Science and Engineering

4190.307: Operating Systems
Midterm Exam. (Fall 2023)

14:00 – 15:15, October 24, 2023. Instructor: Jin-Soo Kim

Student ID: _____

Name: _____

Codename: _____

Q1 (40)		Q4 (70)	
Q2 (30)		Q5 (50)	
Q3 (30)		Q6(50)	
		Total (270)	

I hereby certify that I have neither given nor received aid in answering the questions on this exam. _____
(Signature)

1. 괄호안을 하나의 영어 단어(혹은 이름)로 채우시오. (각 4점)

(1) One of the major practical advances of the time was the introduction of the UNIX operating system, primarily thanks to Ken ( ) (and Dennis Ritchie) at Bell Labs.

(2) Separating the two (i.e., policies and mechanisms) allows one easily to change policies without having to rethink the mechanism and is thus a form of ( ), a general software design principle.

(3) A process list contains information about all processes in the system. Each entry is found in what is sometimes called a process ( ) block, which is really just a structure that contains information about a specific process.

(4) Virtually all modern schedulers are ( ), and quite willing to stop one process from running in order to run another.

(5) After a time, because machines were expensive, people began to share machines more effectively. Thus the era of ( ) was born, in which multiple processes were ready to run at a given time, and the OS would switch between them, for example when one decided to perform an I/O.

(6) One major goal of a virtual memory (VM) system is ( ). The OS should implement virtual memory in a way that is invisible to the running program.

(7) Introduced in the first time-sharing machines of the late 1950's is a simple idea referred to as base and bounds; the technique is also referred to as dynamic ( ).

(8) With a multi-level structure, we add a level of ( ) through use of the page directory, which points to pieces of the page table.

(9) One type of historical information a page-replacement policy could use is ( ); if a page has been accessed many times, perhaps it should not be replaced as it clearly has some value.

(10) Modern systems add some tweaks to straightforward LRU approximations like clock; for example, ( ) resistance is an important part of many modern algorithms, such as ARC.

2. 다음은 xv6/RISC-V 상에서 **ecall** 명령어가 수행되어 커널의 system call handler가 불리기까지 행해져야 할 일들을 순서없이 나열한 것이다.

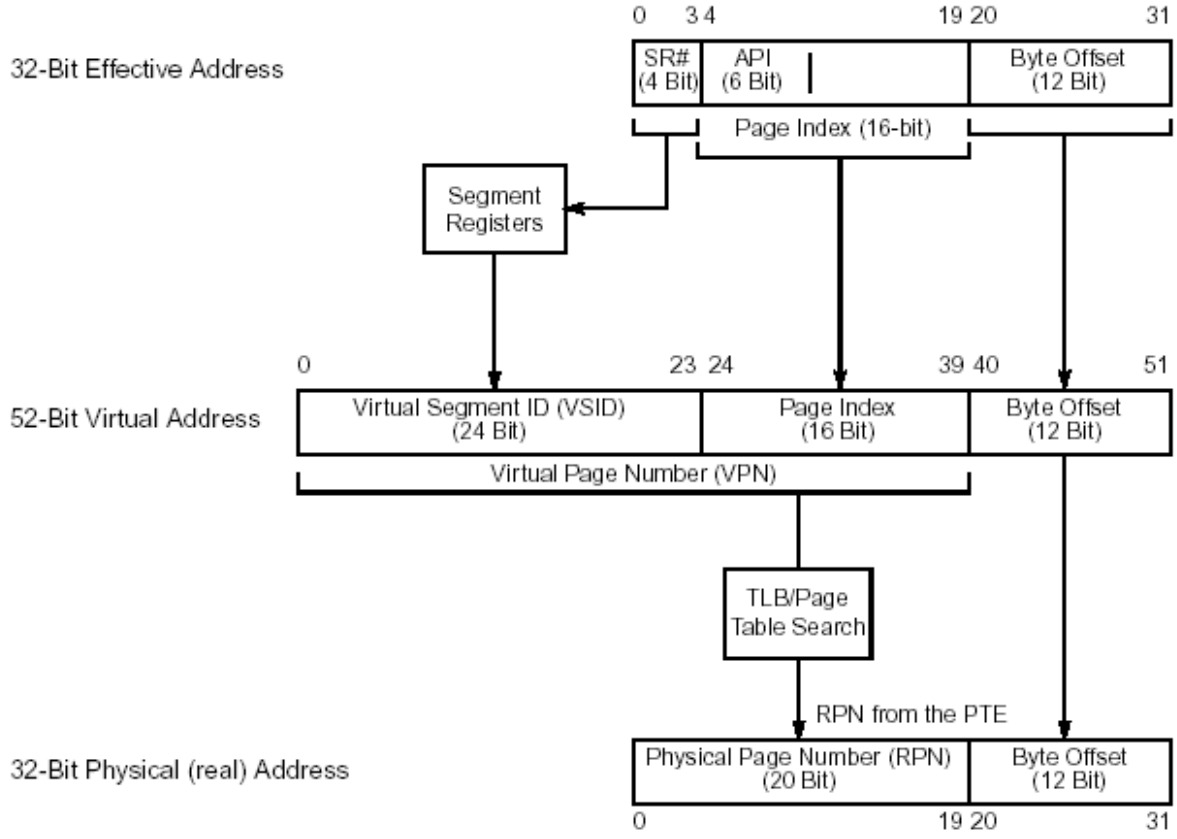
A. Change the CPU's execution mode from user mode to supervisor mode
B. Save the value of the **PC** register to the **SEPC** register
C. Set the **SP** register to point to the top of the kernel stack
D. Save the values of all registers in the trap frame
E. Change the value of the **PC** register to the value of the **STVEC** register
F. Change the value of the **SATP** register to the address of the kernel page table
G. Disable interrupts
H. Record in the **SCAUSE** register that a trap occurred due to a system call

(1) A-H 중에서 **ecall** 명령어가 수행되었을 때 RISC-V CPU에 의해 하드웨어적으로 한 번에 행해지는 항목들은 어떤 것들인지 모두 나열하시오. (20점)

(2) 위의 (1)번 문제에서 답한 항목들을 제외하고 나머지의 항목들이 xv6 상에서 어떤 순서대로 행해지는지 나열하시오. (10점)

3. User address space와 kernel address space가 분리되어 있는 일반적인 OS에서 trap이 발생하였을 경우 user mode에서 사용하던 레지스터의 값들은 kernel stack이나 trap frame 등에 저장된다. 그러나 이러한 용도로 user stack을 사용할 수는 없는가? 즉, trap으로 kernel mode에 진입한 후 user 프로세스의 stack에 레지스터 값들을 저장하였다가 다시 user mode로 돌아갈 해당 user stack으로부터 레지스터를 복구하는 방법이다. 이러한 방법을 사용할 경우의 장단점에 대해서 논하시오. (30점)

4. 아래 그림은 segmentation과 paging을 함께 쓰는 32-bit PowerPC 프로세서에서 주소 변환이 이루어지는 과정을 보인 것이다. Effective address는 PowerPC 프로세서 상에서 load/store 명령이나 instruction fetch 등을 위해 사용하는 주소를 의미한다. 32-bit effective address는 segmentation을 통해 52-bit virtual address로 변환되고, 다시 paging을 통해 최종적으로 32-bit physical (real) address로 변환된 후 메모리를 접근하게 된다.



(1) 위와 같은 32-bit PowerPC 구조에서 page size는 얼마인가? (5점)

(2) 하나의 프로세스가 사용할 수 있는 segment의 개수는 최대 몇 개인가? (5점)

(3) 한 segment의 최대 크기는 얼마인가? (5점)

(4) 위와 같은 32-bit PowerPC 프로세서에 최대로 장착할 수 있는 physical 메모리의 크기는 얼마인가? (5점)

(5) 만일 한 단계의 linear한 page table을 사용한다고 하면, 하나의 프로세스를 위해 필요한 page table의 크기는 얼마인가? 하나의 PTE(page table entry)는 4바이트를 차지한다고 가정한다. (5점)

(6) Multi-level page table을 사용할 경우, 각 단계의 page table 크기가 page size에 맞도록 설계한다면 총 몇 단계의 page table이 필요한가? 위 (5)번과 같은 linear page table은 한 단계만 있는 것으로 본다. (5점)

(7) 하나의 segment를 둘 이상의 프로세스들이 공유할 수 있지만, 시스템에서 사용되는 segment들이 virtual address space 상에서 서로 overlap 되지는 않는다고 가정하자. 이러한 경우 TLB는 언제 flush 되어야 하는가? 이유를 함께 설명하시오. (20점)

(8) 앞의 그림에 보이는 바와 같이 PowerPC의 TLB는 virtual-to-physical mapping을 가지고 있기 때문에, effective address가 virtual address로 변환된 이후에 접근이 가능하다. 만일 TLB가 effective-to-physical mapping을 가지고 있다면, virtual address로의 변환없이 TLB를 접근하는 것이 가능하여 주소 변환의 성능을 높일 수 있을 것이다. 즉, effective address를 이용하여 TLB를 검색하고, TLB는 해당 page에 해당하는 PTE(page table entry)를 캐싱하고 있으면 TLB hit가 나는 경우 바로 physical address로 변환할 수 있다. 이러한 TLB를 사용할 경우 어떤 문제점들이 있을지 이유와 함께 설명하시오. (20점)

5. 다음은 어떤 프로세스가 참조하는 virtual page의 트레이스를 시간순으로 추출한 것이다. 편의상, 참조되는 virtual page number (VPN)를 A, B, C, ... 와 같은 알파벳으로 표시하기로 한다. 초기에 physical memory는 모두 비어 있고, 각각의 VPN은 서로 다른 page frame에 대응한다고 가정한다. 다음 물음에 답하시오.

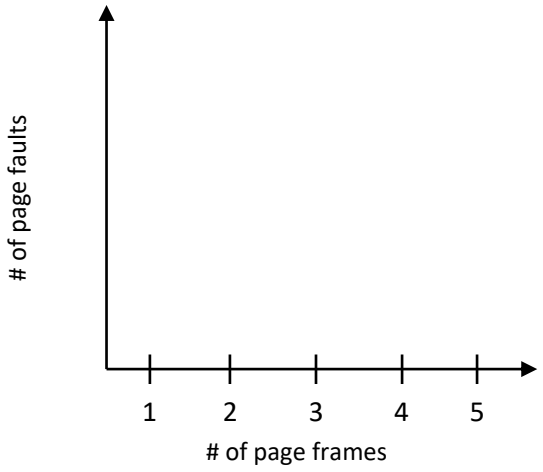
A	B	E	A	E	C	D	E	C	E	A	B	C	B	A	D	C	A	B	A

(1) Physical page frame의 수가 3이라고 할 때, OPT (Belady's optimal algorithm) 교체 정책을 사용했을 경우 발생하는 page fault의 횟수는? (10점)

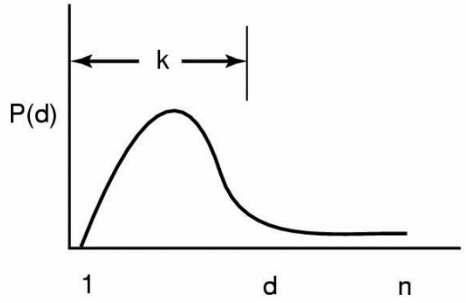
(2) 메모리 참조를 순서에 따라, t = 0, 1, 2, ...와 같이 일련번호를 붙이고, 각각의 순간에 참조되는 페이지를 $R(t)$라고 하자. 어느 순간 t에 참조되는 페이지 $R(t)$가 처음 참조되는 것이 아닐 경우, 바로 직전에 $R(t)$가 참조된 시간부터 현재 t까지 프로세스에 의해 참조된 서로 다른 페이지($R(t)$를 포함)의 수를 $D(t)$라고 정의한다. 만일 $R(t)$가 처음 참조되는 페이지라면, $D(t)$의 값은 ∞이다. $D(t)$의 값을 stack distance 혹은 reference distance라고 부른다. 위에 보인 메모리 참조 트레이스에서 각 순간 메모리 참조에 대한 $D(t)$ 값을 구하시오. (10점)

t	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
$R(t)$	A	B	E	A	E	C	D	E	C	E	A	B	C	B	A	D	C	A	B	A
$D(t)$																				

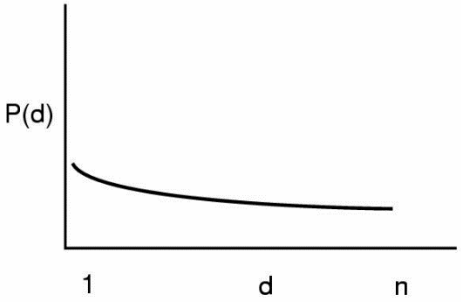
(3) Stack distance $D(t)$는 과거에 동일한 페이지를 접근한 때로부터 현재 다시 해당 페이지를 접근할 때까지 참조된 서로 다른 페이지의 수를 나타낸다. 이와 같은 성질을 이용하여, page frame의 수 m이 주어졌을 때 LRU 교체 정책 하에서 page fault 횟수 $PF_{LRU}(m)$를 구하는 방법을 $D(t)$를 이용하여 제시하시오. 또한, $PF_{LRU}(m)$를 이용하여 page frame의 수가 1에서 5까지 증가할 경우 발생하는 page fault 횟수를 구하고 그래프로 그리시오. (10점)



(4) *P(d)*를 어떤 임의의 순간에 행하는 메모리 참조의 stack distance가 *d*일 확률이라고 정의한다. 어떤 두 개의 프로세스 A, B의 메모리 참조 트레이스에 대해 stack distance를 구하고, 이를 바탕으로 *P(d)*의 그래프를 그렸더니, 다음과 같았다. 두 프로세스는 메모리 참조 특성면에서 어떤 차이가 있는가? 이것이 page replacement와 관련하여 의미하는 바는 무엇인가? (20점)



(a) Process A



(b) Process B

6. 다음은 xv6에서 physical memory에 할당되는 주요한 변수 혹은 data structure를 나타낸 것이다. 각각의 용도를 간단히 설명하고, 해당 변수나 data structure가 프로세스마다 하나씩 할당되는지 (per-process), CPU core마다 하나씩 할당되는지 (per-core), 아니면 시스템에 하나만 할당되는지 (per-system) 해당되는 부분에 V 표시하시오. (50점)

Xv6 Variables or Data structures	사용 용도	Per-process?	Per-core?	Per-system?
`pagetable_t kernel_pagetable`				
`char trampoline[]`				
`struct trapframe`				
`struct proc`				
`struct cpu`				
`uint ticks`				
`static uint64 (*syscalls[])(void)`				